

Scaffold

2024年4月22日 星期一 23:06

Scaffold 是 Material 库中提供的页面脚手架，它提供了默认的导航栏，标题和包含主屏幕 widget 树与“组件树”或“部件树”相同的 body 属性，组件树可以很复杂，路由默认都是通过 Scaffold 创建。

Color

2024年4月23日 星期二 23:46

```
return Scaffold(  
  appBar: AppBar(  
    // ARGB A: 代表后面的RGB颜色的浓度  
    // backgroundColor: Color.fromARGB(255, 255, 0, 0),  
    //RGBO O: 代表前面RGB颜色的透明度  
    // backgroundColor: Color.fromRGBO(255, 0, 0, 0.5),  
    // 使用十六进制的颜色  
    // backgroundColor: Color(0xFFFF0000),  
    // 某个颜色中颜色占有的比例  
    backgroundColor: Colors.red[500],  
    title: Text("Small Ding"),  
  ), // AppBar
```

Color.fromARGB: A代表后面RGB颜色的浓度，类似修图工具中的饱和度

Color.fromRGBO: O代表前面RGB颜色的透明度

Color(十六进制): 其中可以直接填写十六进制的颜色

Colors.red[数值]: 其中的数值代表某个颜色的颜色占比大小，例如这里就是红色的颜色占比

Margin / Padding

2024年4月24日 星期三 23:05

Margin 和 padding 基本上通用的几种方法，他们都采用 `EdgeInsets` 类

采用 `EdgeInsets` 类

```
1 // 1. 所有方向均使用相同数值的填充。  
2 all(double value)  
3 // 示例: 4个方向各添加16像素补白  
4 padding: EdgeInsets.all(16.0)  
5  
6 // 2. 分别指定四个方向的不同填充  
7 fromLTRB(double left, double top, double right, double bottom)  
8 // 示例:  
9 padding: const EdgeInsets.fromLTRB(10,20,30,40)  
10  
11 // 3. 设置具体某个方向的填充(可以同时指定多个方向)  
12 only({left, top, right, bottom })  
13 // 示例: 在左边添加8像素补白  
14 padding: const EdgeInsets.only(left: 8.0),  
15  
16 // 4. 设置对称方向的填充  
17 // vertical: 针对垂直方向top、bottom  
18 // horizontal: 针对横向方向left、right  
19 symmetric({ vertical, horizontal })  
20 // 示例: 垂直方向上下各添加8像素补白  
21 padding: const EdgeInsets.symmetric(vertical: 8.0)
```

StatelessWidget

2024年4月21日 星期日 21:45

StatelessWidget继承了Widget类

它用于不需要维护状态的场景，它通常在 build 方法中嵌套其他的 widget 来构建UI，在构建过程中会递归其嵌套的widget。

```
class Echo extends StatelessWidget {
  const Echo({
    Key? key,
    required this.text,
    this.backgroundColor = Colors.grey, //默认为灰色
  }):super(key:key);

  final String text;
  final Color backgroundColor;

  @override
  Widget build(BuildContext context) {
    return Center(
      child: Container(
        color: backgroundColor,
        child: Text(text),
      ),
    );
  }
}
```

State

2024年6月14日 星期五 22:57

一个 `StatefulWidget` 类会对应一个 `State` 类, `State` 表示与其对应的 `StatefulWidget` 要维护的状态, `State` 中的保存的状态信息可以:

1. 在 widget 构建时可以被同步读取。
2. 在 widget 生命周期中可以被改变, 当 `State` 被改变时, 可以手动调用其 `setState()` 方法通知 Flutter 框架状态发生改变, Flutter 框架在收到消息后, 会重新调用其 `build` 方法重新构建 widget 树, 从而达到更新 UI 的目的。

`State` 中有两个常用属性:

1. `widget`, 它表示与该 `State` 实例关联的 widget 实例, 由 Flutter 框架动态设置。注意, 这种关联并非永久的, 因为在应用生命周期中, UI 树上的某一个节点的 widget 实例在重新构建时可能会变化, 但 `State` 实例只会在第一次插入到树中时被创建, 当在重新构建时, 如果 widget 被修改了, Flutter 框架会动态设置 `State.widget` 为新的 widget 实例。
2. `context`。 `StatefulWidget` 对应的 `BuildContext`, 作用同 `StatelessWidget` 的 `BuildContext`。

#

initState

类似 uni 中的 `onLoad` 函数当 widget 第一次插入到 widget 树中会被调用, 对于每一个 `State` 对象, Flutter 框架只会调用一次该回调, 通常在该回调中做一些一次性的操作, 比如状态初始化、订阅子树的事件通知等

注意: 不能在该回调中调用 `BuildContext.dependOnInheritedWidgetOfExactType` (该方法用于在 widget 树上获取离当前 widget 最近的一个父级 `InheritedWidget`。

原因是在初始化完成后, widget 树中的 `InheritFrom widget` 也可能会有变化, 所以正确的做法应该在 `build()` 方法或 `didChangeDependencies()` 中调用它。

didChangeDependencies ()

当 `State` 对象的依赖发生变化时会被调用; 例如: 在之前 `build()` 中包含了一个 `InheritedWidget` (第七章介绍), 然后在之后的 `build()` 中 `Inherited widget` 发生了变化, 那么此时 `InheritedWidget` 的子 widget 的 `didChangeDependencies()` 回调都会被调用。典型的场景是当系统语言 `Locale` 或应用主题改变时, Flutter 框架会通知 widget 调用此回调。需要注意, 组件第一次被创建后挂载的时候 (包括重创建) 对应的

`didChangeDependencies` 也会被调用。

`build ()`

它主要是用于构建 widget 子树的，会在如下场景被调用：

1. 在调用 `initState()` 之后。
2. 在调用 `didUpdateWidget()` 之后。
3. 在调用 `setState()` 之后。
4. 在调用 `didChangeDependencies()` 之后。
5. 在 State 对象从树中一个位置移除后（会调用 `deactivate`）又重新插入到树的其他位置之后。

`reassemble ()`

这个回调是专门针对调试提供的，只会在热重载的时候调用，在 Release 中不会被调用，所以需要注意重要代码的使用。

`didUpdateWidget ()`

这个回调只会在更新了 widget 树内容才会被调用，就是说只有内容发生改变时才会。Flutter 框架会调用 `widget.canUpdate` 来检测 widget 树中同一位置的新旧节点，然后觉得是否需要更新，如果 `widget.canUpdate` 返回 `true` 则会调用，正如之前所述，`widget.canUpdate` 会在新旧 widget 的 `key` 和 `runtimeType` 同时相等时会返回 `true`，也就是说在在新旧 widget 的 `key` 和 `runtimeType` 同时相等时 `didUpdateWidget()` 就会被调用。

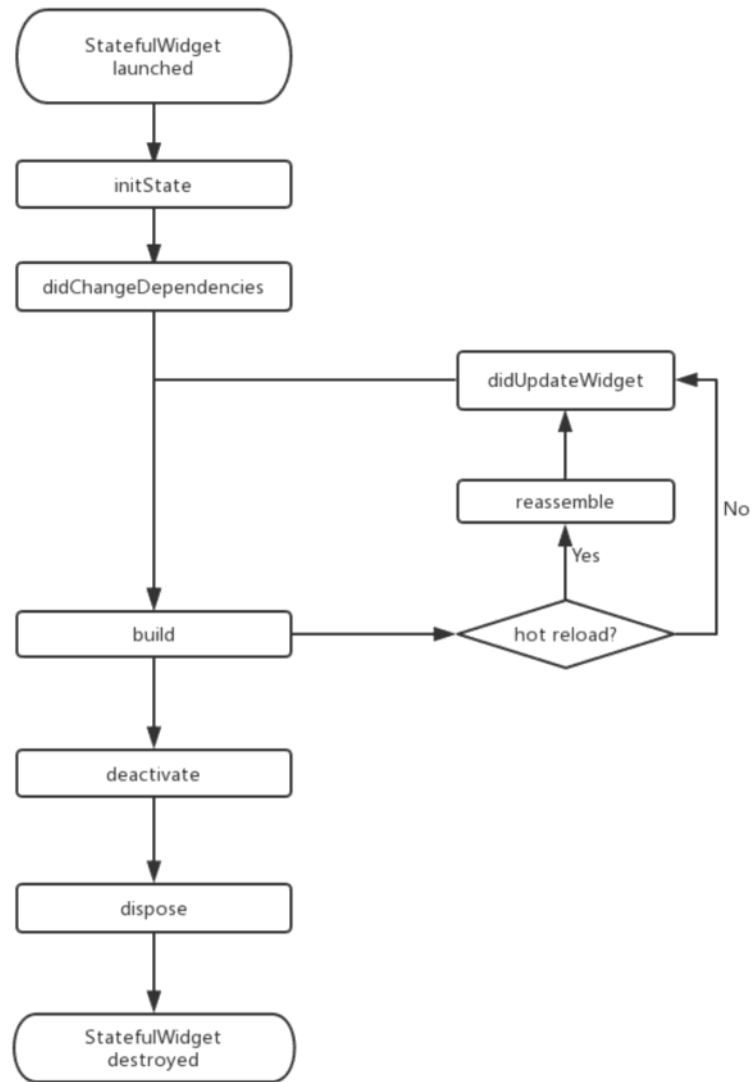
`deactivate ()`

当 State 对象从树中被移除时，会调用此回调。在一些场景下，Flutter 框架会将 State 对象重新插到树中，如包含此 State 对象的子树在树的一个位置移动到另一个位置时（可以通过 `GlobalKey` 来实现）。如果移除后没有重新插入到树中则紧接着会调用 `dispose()` 方法。

也就是说如果 State 对象从树中被移除时如果没有移动到另一个位置（移动位置可以 `GlobalKey` 来实现）那么就会触发 `dispose ()` 方法，否则就会触发这个方法。

`dispose ()`

当 State 对象从树中被永久移除时调用；通常在此回调中释放资源。



注意：在继承StatefulWidget重写其方法时，对于包含@mustCallSuper标注的父类方法，都要在子类方法中调用父类方法。 这句话没怎么懂

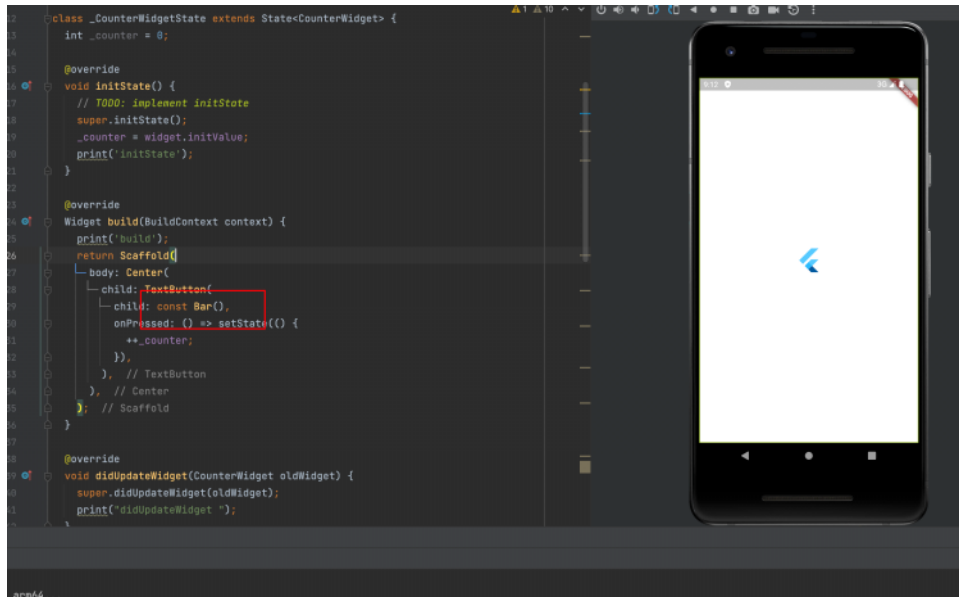
#

State状态提升

2024年6月25日 星期二 20:58

Flutter 在修改某个状态后会build 整个 widget，也就是会刷新整个页面，但对于某些不需要改变的内容这样就会显得没有必要

解决这种问题的其中一个方法就是使用 const

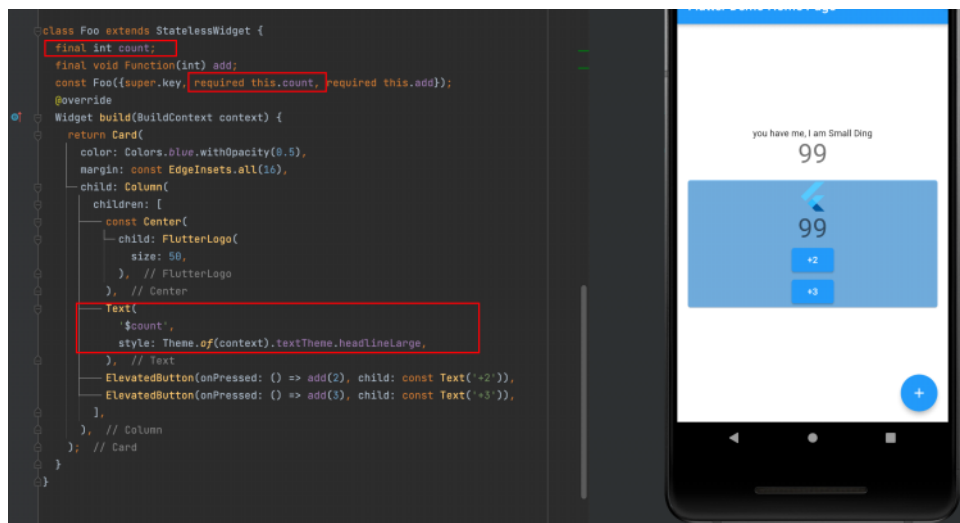
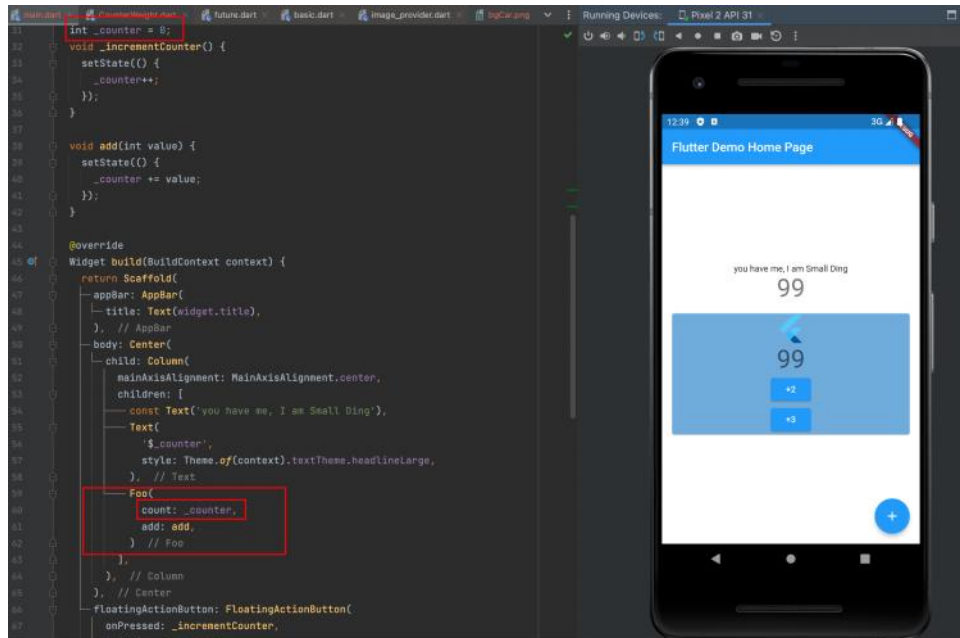


用 const 来表明这个组件是不可变的 这样build就不会再重新构建，但有个问题就是这个组件不能传入任何参数或者任何会改变组件的操作，因为它被const标识了

State 访问和修改外部状态

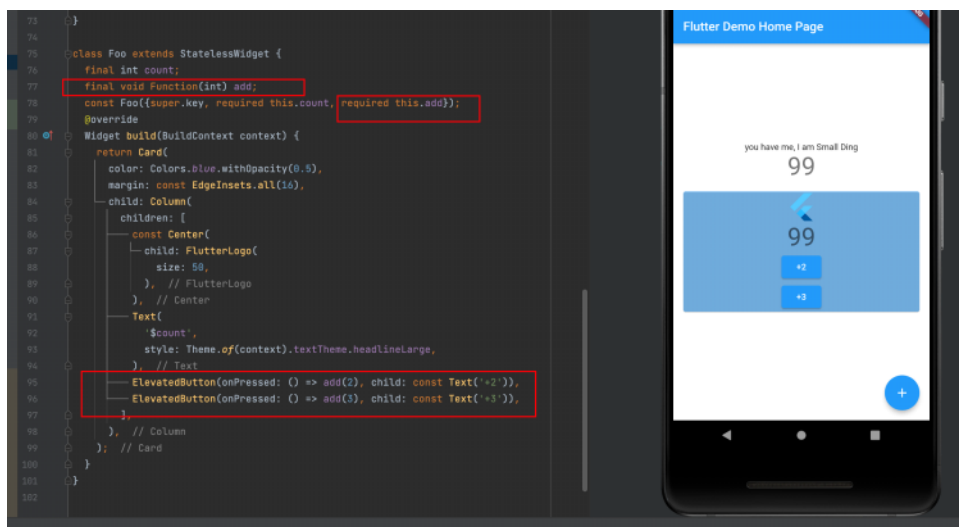
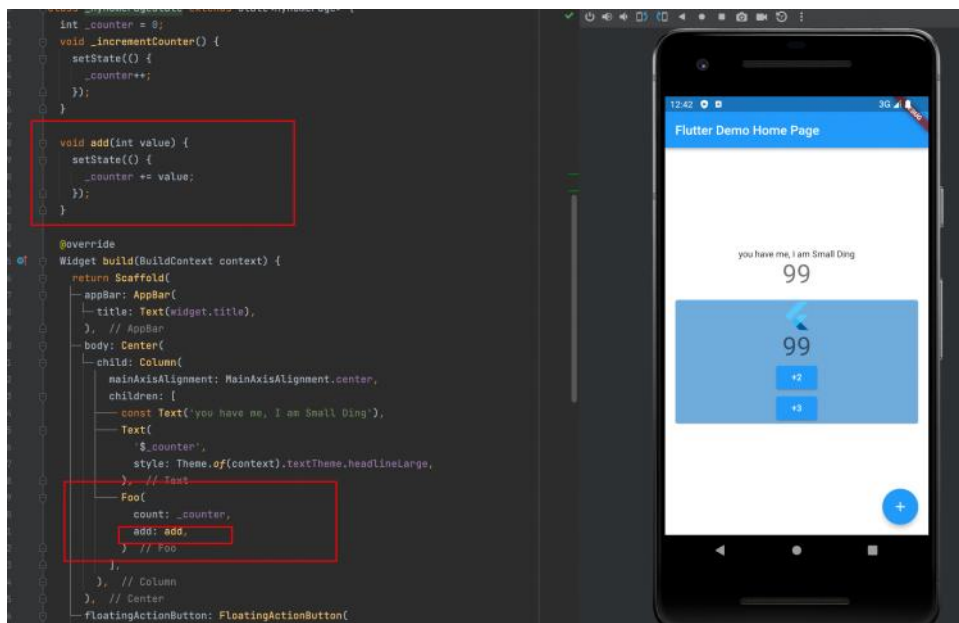
2024年6月26日 星期三 00:38

访问外部状态



创建B组件并声明出一个count 并设为必填，在A组件中通过调用B组件并传入数值从而获取对应状态

修改外部状态



修改外部状态与获取外部状态类似，但声明的是一个方法，这个案例中是需要传递参数的，所以在Function (int) 中需要设置类型，然后在按钮中的onPressed直接调用该方法，该方法也是在A组件中声明的，

Final void Function () add

当方法没有传递的参数和返回值时就是用这种方式命名，还可以用 VoidCallBack ()

例如：final VoidCallBack add 来声明，这两种方式是同等的

VoidCallBack () 源码

```
part of dart.ui;

/// Signature of callbacks that have no arguments and return no data.
typedef VoidCallback = void Function();

/// Signature for [PlatformDispatcher.onBeginFrame].
typedef FrameCallback = void Function(Duration duration);

|
/// Signature for [PlatformDispatcher.onReportTimings].
///
/// {@template dart.ui.TimingsCallback.list}
/// The callback takes a list of [FrameTiming] because it may not be
/// immediately triggered after each frame. Instead, Flutter prints the
```

StatefulWidget

2024年4月22日 星期一 22:57

Stateful widget 可以拥有状态，这些状态在 widget 生命周期中是可以变的

它是由 State 类和 StatefulWidget

Stateful widget 本身是不变的，但是 State 类中持有的状态在 widget 生命周期中可能会发生变化

在 widget 树中获取State对象

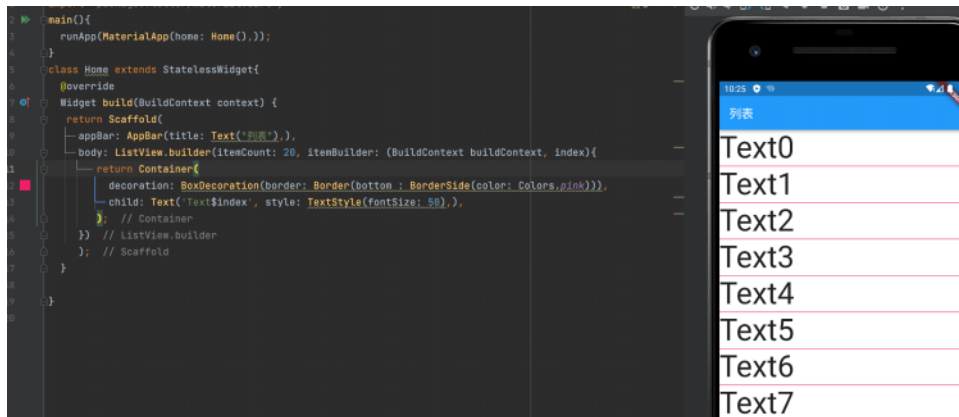
2024年6月18日 星期二 22:44

StatefulWidget 的具体逻辑都在其 State 中

ListView.builder

2024年6月4日 星期二 22:24

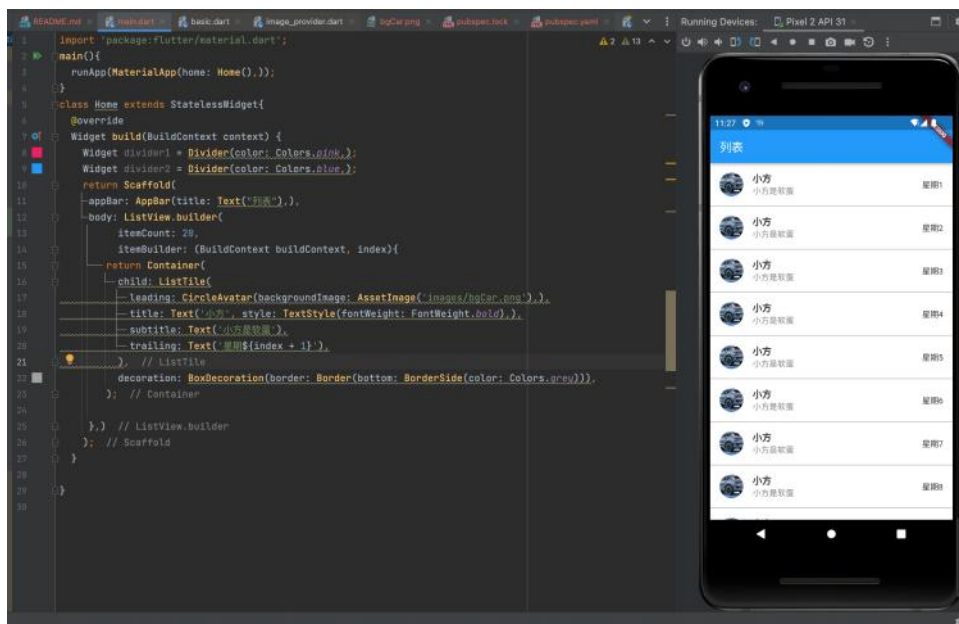
ListView.builder 适合列表项比较多或者列表项不确定的情况

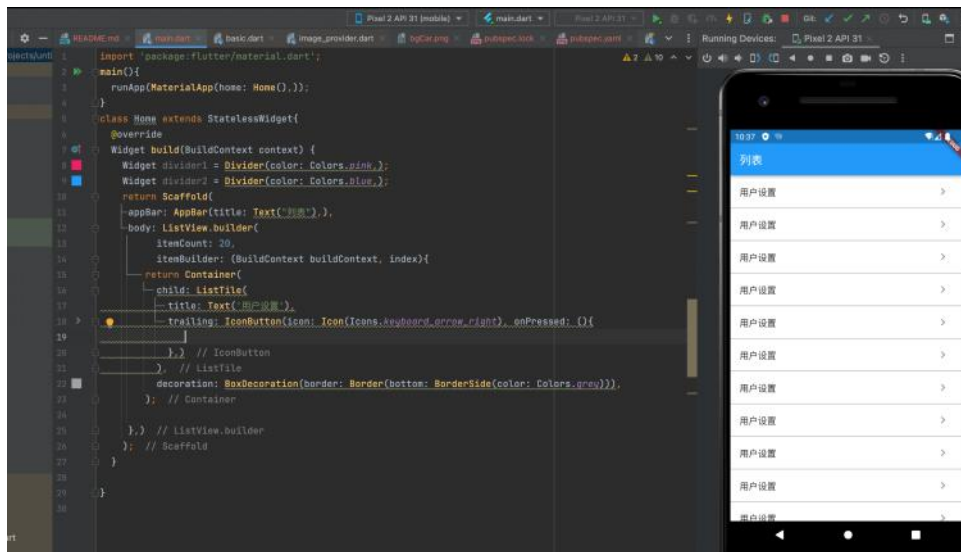


itemBuilder: 它是列表项的构建器，类型为IndexedWidgetBuilder，返回值为一个widget。当列表滚动到具体的 index 位置时，会调起该构建器构建列表项。

itemCount: 列表项的数量，如果为null，则为无限列表。

itemExtent: 可强制高度

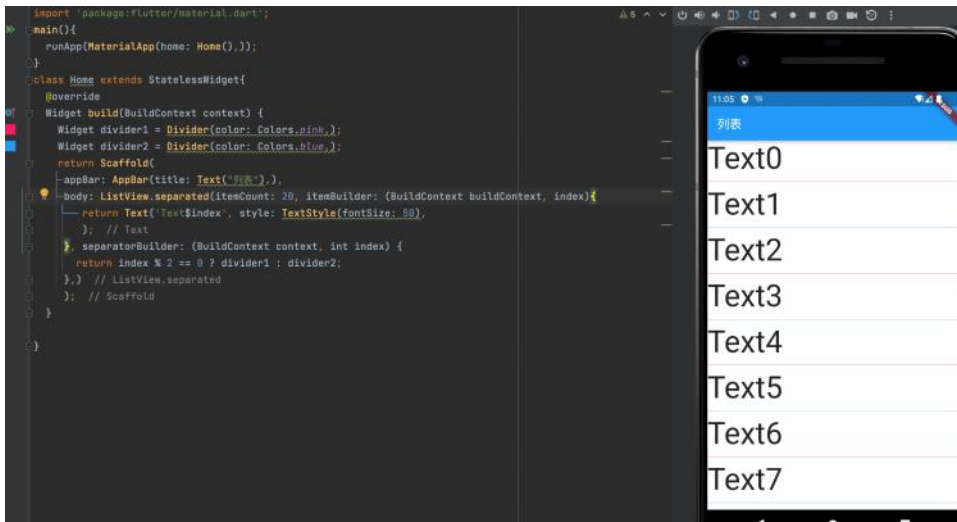




ListView.separated

2024年6月4日 星期二 23:04

ListView.separated 可以在生成的列表项之间添加一个分割组件，它比ListView.builder 多了一个 separatorBuilder 参数，该参数是一个分割组件生成器



对话框

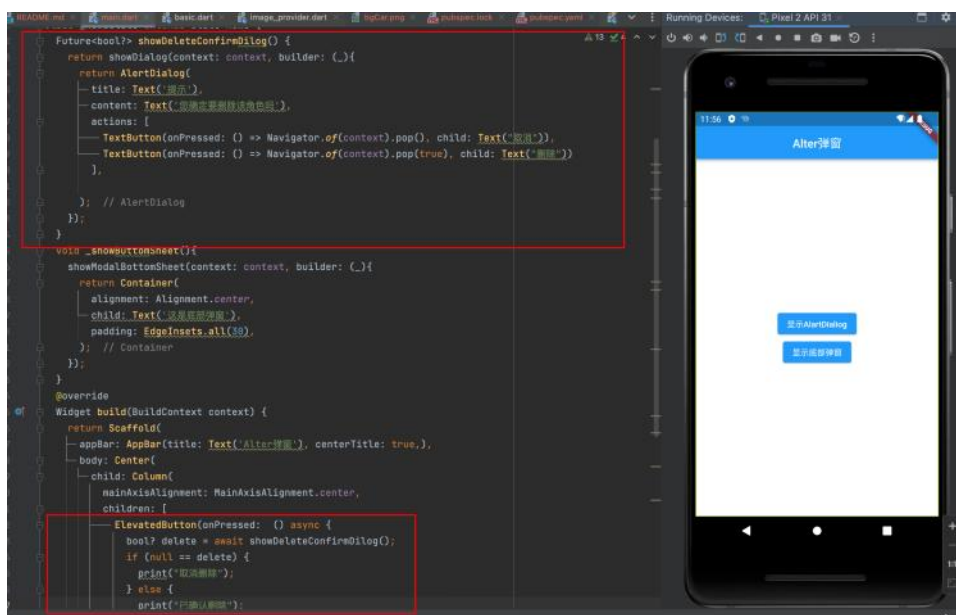
2024年6月5日 星期三 23:02

对话框本质上越是一个 UI 布局类似现实一个画板你可以在这个画板里面协商你需要的东西

1. AlertDialog

```
1  const AlertDialog({
2    Key? key,
3    this.title, //对话框标题组件
4    this.titlePadding, // 标题填充
5    this.titleTextStyle, //标题文本样式
6    this.content, // 对话框内容组件
7    this.contentPadding = const EdgeInsets.fromLTRB(24.0, 20.0, 24.0, 24.0), //内容的填充
8    this.contentTextStyle, // 内容文本样式
9    this.actions, // 对话框操作按钮组
10   this.backgroundColor, // 对话框背景色
11   this.elevation, // 对话框的阴影
12   this.semanticLabel, //对话框语义化标签(用于读屏软件)
13   this.shape, // 对话框外形
14 })
```

AlertDialog 只能写在 StatefulWidget 中

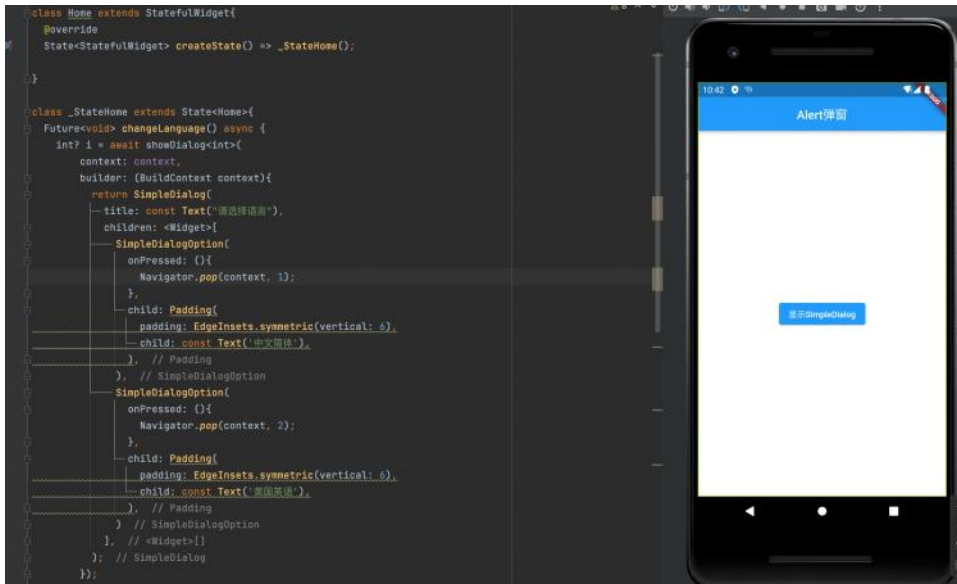


这里 Future 返回一个布尔值 当点击了取消时返回的是null 点击删除时返回的是true

注意：如果AlertDialog的内容过长，内容将会溢出，这在很多时候可能不是我们期望的，所以如果对话框内容过长时，可以用SingleChildScrollView将内容包裹起来。

2. SimpleDialog

SimpleDialog也是Material组件库提供的对话框，它会展示一个列表，用于列表选择的场景。下面是一个选择APP语言的示例

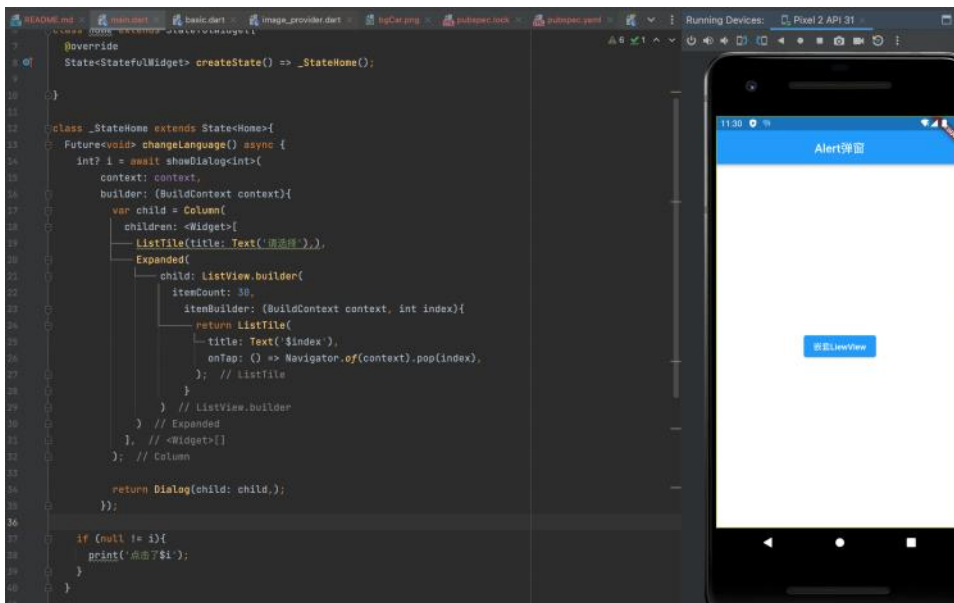


3. Dialog

实际上以上两种对话框都使用了 `Dialog` 类。由于 `AlertDialog` 和 `SimpleDialog` 中使用了 `Int.rinsicWidth` 来尝试通过自组件的实际尺寸来调整自身尺寸，这就导致他们的子组件不能是延迟加载模型的组件，如：`ListView`、`GridView`、`CustomScrollView` 等，比如下面这样的代码就会报错。

```
1  AlertDialog(  
2    content: ListView(  
3      children: ...//省略  
4    ),  
5  );
```

如果就是需要嵌套一个 `ListView` 那么可以直接使用 `Dialog` 类



上面的这些例子里在调用showDialog的时候在builder中都是构建了这三个对话框的组件的一种，但是不仅仅可以用这三种，比如：

```
1 // return Dialog(child: child)
2 return UnconstrainedBox(
3   constrainedAxis: Axis.vertical,
4   child: ConstrainedBox(
5     constraints: BoxConstraints(maxWidth: 280),
6     child: Material(
7       child: child,
8       type: MaterialType.card,
9     ),
10  ),
11 );
```

上面代码运行后可以实现一样的效果。现在我们总结一下：
AlertDialog、SimpleDialog以及Dialog是Material组件库提供的三种对话框，旨在帮助开发者快速构建出符合Material设计规范的对话框，但读者完全可以自定义对话框样式，因此，我们仍然可以实现各种样式的对话框，这样即带来了易用性，又有很强的扩展性。

但是目前没动这个代码的意思

对话框打开动画及遮罩

2024年6月5日 星期三 23:02

对话框可以分为内部样式和外部样式，内部样式指对话框中显示的具体内容，比如上一个页面所写的，这页则是外部样式

showDialog方法是 Material 组件库中提供的一个打开 Material 风格对话框的方法。打开普通风格的对话框（非Material风格）Flutter 提供了一个 showGeneralDialog 方法，签名如下：

```
1 Future<T?> showDialog<T>({  
2   required BuildContext context,  
3   required RoutePageBuilder pageBuilder, //构建对话框内部UI  
4   bool barrierDismissible = false, //点击遮罩是否关闭对话框  
5   String? barrierLabel, // 语义化标签(用于读屏软件)  
6   Color barrierColor = const Color(0x80000000), // 遮罩颜色  
7   Duration transitionDuration = const Duration(milliseconds: 200), // 对话框打开  
8   RouteTransitionsBuilder? transitionBuilder, // 对话框打开/关闭的动画  
9   ...  
10 })
```

实际上showDialog方法是showGeneralDialog的一个封装，定制了Material风格对话框的遮罩颜色和动画。

Material风格对话框打开/关闭动画是一个 Fade（渐隐渐显）动画，如果想换一种缩放动画可以使用 transitionBuilder 来定义

```

1  Future<T?> showCustomDialog<T>({
2      required BuildContext context,
3      bool barrierDismissible = true,
4      required WidgetBuilder builder,
5      ThemeData? theme,
6  }) {
7      final ThemeData theme = Theme.of(context, shadowThemeOnly: true);
8      return showGeneralDialog(
9          context: context,
10         pageBuilder: (BuildContext buildContext, Animation<double> animation,
11             Animation<double> secondaryAnimation) {
12             final Widget pageChild = Builder(builder: builder);
13             return SafeArea(
14                 child: Builder(builder: (BuildContext context) {
15                     return theme != null
16                         ? Theme(data: theme, child: pageChild)
17                         : pageChild;
18                 }),
19             );
20         },
21         barrierDismissible: barrierDismissible,
22         barrierLabel: MaterialLocalizations.of(context).modalBarrierDismissLabel,
23         barrierColor: Colors.black87, // 自定义遮罩颜色
24         transitionDuration: const Duration(milliseconds: 150),
25         transitionBuilder: _buildMaterialDialogTransitions,
26     );
27 }
28
29 Widget _buildMaterialDialogTransitions(
30     BuildContext context,
31     Animation<double> animation,
32     Animation<double> secondaryAnimation,
33     Widget child) {
34     // 使用缩放动画
35     return ScaleTransition(
36         scale: CurvedAnimation(
37             parent: animation,
38             curve: Curves.easeOut,
39         ),
40         child: child,
41     );

```

现在，我们使用 `showCustomDialog` 打开文件删除确认对话框，代码如下：

```
1    ... //省略无关代码
2    showCustomDialog<bool>(  
3        context: context,  
4        builder: (context) {  
5            return AlertDialog(  
6                title: Text("提示"),  
7                content: Text("您确定要删除当前文件吗?"),  
8                actions: <Widget>[  
9                    TextButton(  
10                       child: Text("取消"),  
11                       onPressed: () => Navigator.of(context).pop(),  
12                       ),  
13                    TextButton(  
14                       child: Text("删除"),  
15                       onPressed: () {  
16                           // 执行删除操作  
17                           Navigator.of(context).pop(true);  
18                       },  
19                    ),  
20                ],  
21            );  
22        },  
23    );
```

对话框实现原理

2024年6月8日 星期六 23:33

这边的原理由于基础不行，暂时搁置 .

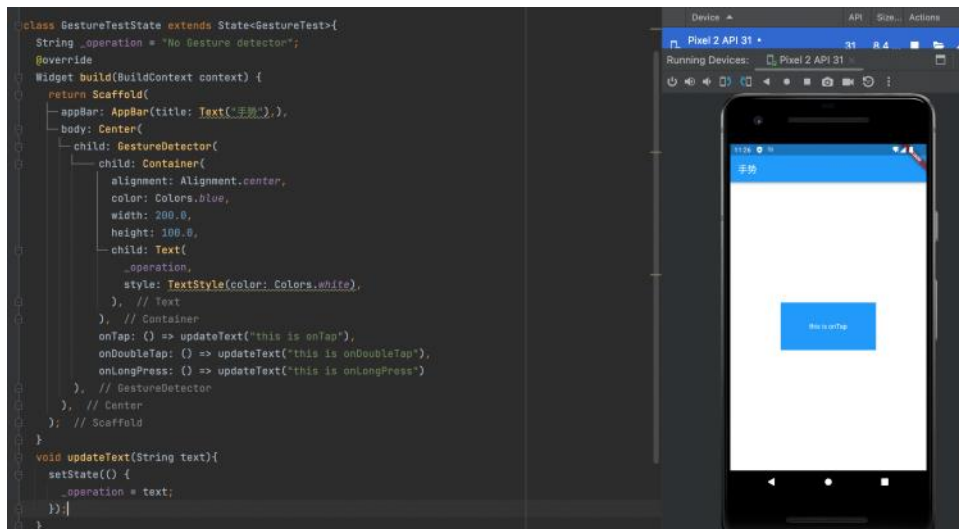
GestureDetector

2024年4月25日 星期四 23:24

这是一个用于手势识别的功能性组件，可以通过它来识别各种手势
它内部封装了 Listener 用于识别语义化的手势

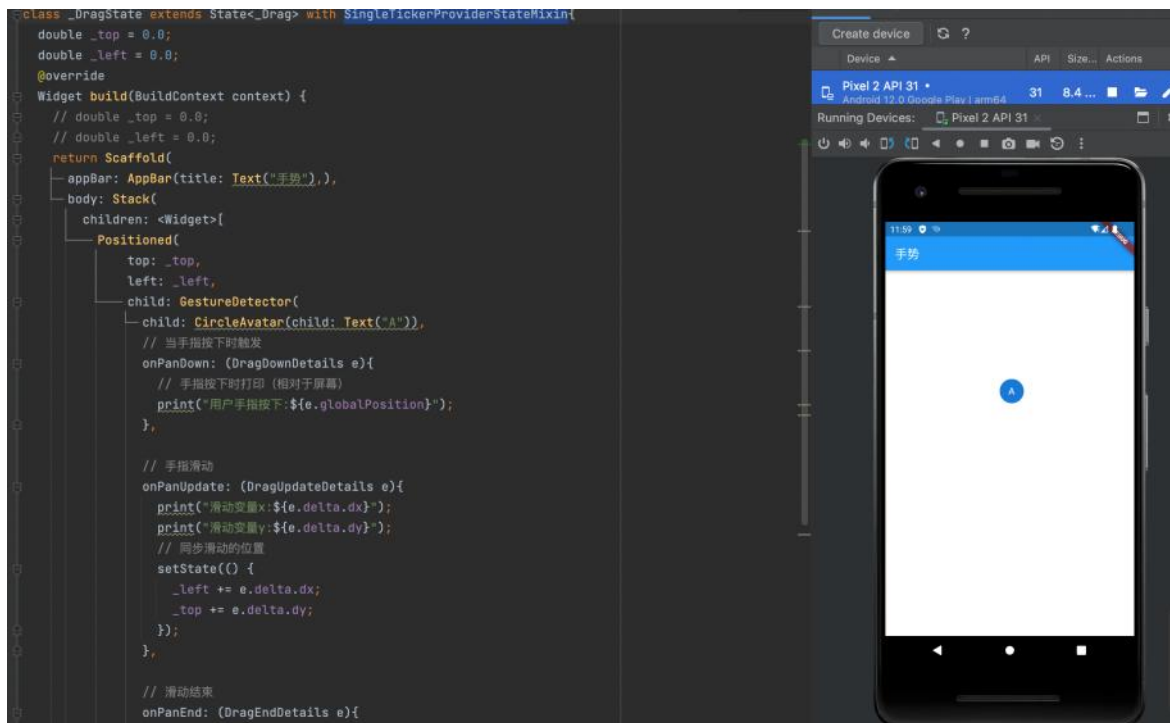
这是点击、双击、长按的案例

注意：当同时监听 onTap 和 onDoubleTap 时当触发 onTap 事件会有200ms的延迟
这是因为用户点击完可能会再次点击触发双击事件，所以会等待一定事件来识别用户的操作

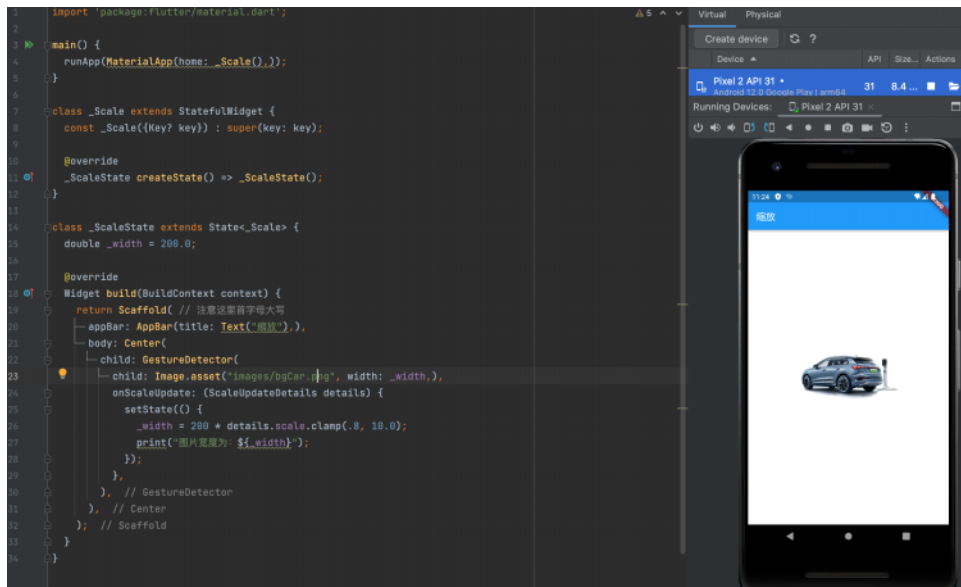


这是长按拖动的案例

在编写这个案例时犯了一个错误：第一次将位置的变量声明在了 `build` 方法中 这会导致每次 `build` 方法被调用时（比如这里状态更新时）这两个变量都会被重新重置为初始化0.0 如图片中的注释部分，实际上需要在 `_DragState` 下将变量声明为状态变量



这是缩放的案例



GestureRecognizer

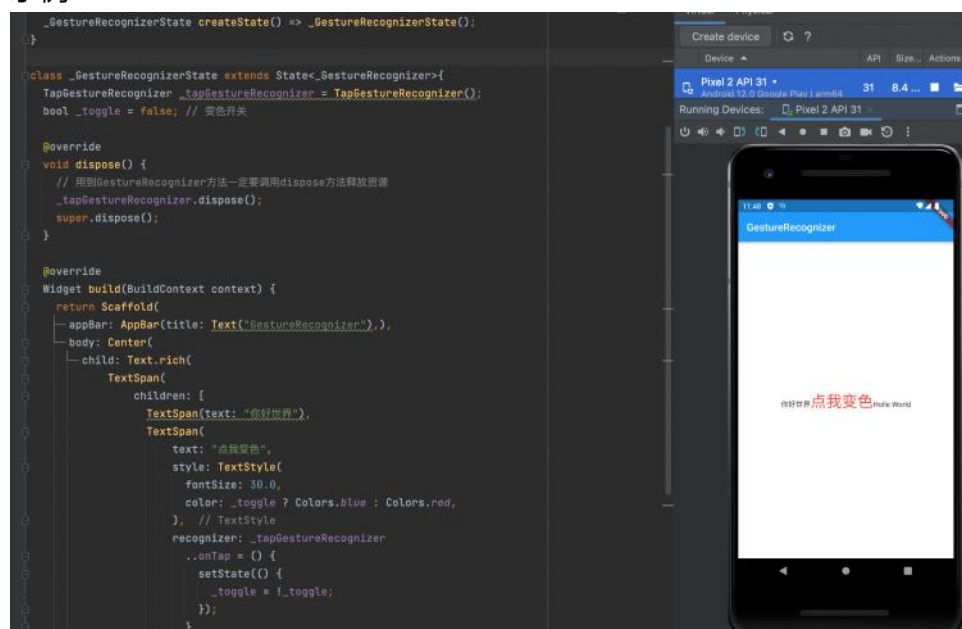
2024年4月27日 星期六 23:48

GestureDetector 内部是使用一个或多个的 GestureRecognizer 来识别各种手势的

GestureRecognizer 的作用就是通过Listener 来将原始指针事件转换为语义手势

GestureDetector 是可以直接接受一个子widget 它是一个抽象类，一种手势的识别器对应一个 GestureRecognizer 的子类，Flutter实现了丰富的手势识别器让我们可以直接使用

示例



Image

2024年4月21日 星期日 21:55

flutter中，使用image加载并显示图片，image的数据源可以是asset、文件、内存以及网络

从asset中加载图片

在工程中目录下创建一个images目录，并放入图片

在pubspec.yaml中的 flutter 部分取消注释如下内容，大概在63行，这里需要特别注意yaml文件对缩进的严格，所以必须严格按照每一层两个空格的方式尽享缩进，此处assets前面应有两个空格。

```
61 # To add assets to your application, add an assets section, like this:
62 assets:
63   - images/
64   # - images/a_dot_ham.jpeg
```

从网络加载图片

```
return Scaffold(
  appBar: AppBar(title: Text("Small Ding"), centerTitle: true, backgroundColor: Colors.red, ),
  body: Image.network('https://media.nature.com/w400/magazine-assets/d41586-024-01037-0/d41586-02
```

CircleAvatar(用户头像)

2024年4月21日 星期日 22:27

这个组件表示用户的圆圈（其实可以理解为头像）

如果 [foregroundImage](#) 失败，则使用 [backgroundImage](#)。如果 [backgroundImage](#) 也失败，则使用 [backgroundColor](#)。

如果 [backgroundImage](#) 为 null，则 [onBackgroundImageError](#) 参数必须为 null。如果 [foregroundImage](#) 为 null，则 [onForegroundImageError](#) 参数必须为 null。

如果头像要具有图像，则应在 backgroundImage 属性中指定图像

```
appBar: AppBar(title: Text("Small Ding"), centerTitle: true, backgroundColor: Colors.red, ),  
body: CircleAvatar(backgroundImage: NetworkImage('https://media.nature.com/w480/magazine-assets/d4t1  
) // Scaffold
```

按钮

2024年4月22日 星期一 22:11

Material 组件库中提供了很多按钮组件：ElevatedButton、TextButton、OutlinedButton等。这些按钮都是直接或间接的对RawMaterialButton 组件的包装定制，所以他们大多数属性都和 RawMaterialButton 一样。

所有Material库中的按钮都有如下相同点

按下时都会有“水波动画”

有一个 onPressed 属性来设置点击回调，当按钮按下时会执行该回调，如果不提供该回调则按钮会处于禁用状态，禁用状态不响应用户点击。

按钮实现比较简单可[查看文档](#)学习。

FadeInImage

2024年4月21日 星期日 22:38

显示占位符图像的图像，而目标图像是加载，然后在加载时淡入新图像

这个小组件可以用于长时间加载的图像，比如从网络上加载的图像，这样图像会比较优美的动画出现在屏幕上，而不是突然出现在屏幕上。

也可以用于当图片加载不出来时的占位图片

```
appBar: AppBar(title: Text("Small Ding"), centerTitle: true, backgroundColor:
body: FadeInImage(
  placeholder: AssetImage('images/bg_car_02.png'),
  image: NetworkImage('https://media.nature.com/w400/magazine-assets/d41586-02'),
), // FadeInImage
); // Scaffold
```

fadeInDuration → 持续时间

图像的淡入动画的持续时间。

最后

fadeOutCurve → 曲线

占位符的淡出动画曲线。

最后

fadeOutDuration → 持续时间

占位符的淡出动画的持续时间。

最后

TextField

2024年5月1日 星期三 22:00

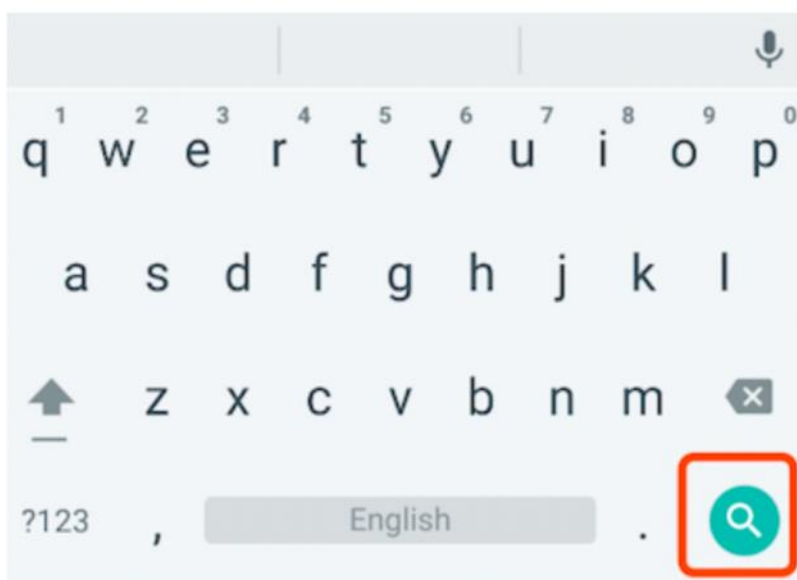
这是文本输入框的一些基础属性

```
1  const TextField({
2    ...
3    TextEditingController controller,
4    FocusNode focusNode,
5    InputDecoration decoration = const InputDecoration(),
6    TextInputType keyboardType,
7    TextInputAction textInputAction,
8    TextStyle style,
9    TextAlign textAlign = TextAlign.start,
10   bool autofocus = false,
11   bool obscureText = false,
12   int maxLines = 1,
13   int maxLength,
14   this.maxLengthEnforcement,
15   ToolbarOptions? toolbarOptions,
16   ValueChanged<String> onChanged,
17   VoidCallback onEditingComplete,
18   ValueChanged<String> onSubmitted,
19   List<TextInputFormatter> inputFormatters,
20   bool enabled,
21   this.cursorWidth = 2.0,
22   this.cursorRadius,
23   this.cursorColor,
24   this.onTap,
25   ...
26 })
```

- **controller**: 编辑框的控制器，通过它可以设置/获取编辑框的内容、选择编辑内容、监听编辑文本改变事件。大多数情况下我们都需要显式提供一个controller来与文本框交互。如果没有提供controller，则TextField内部会自动创建一个。
- **focusNode**: 用于控制TextField是否占有当前键盘的输入焦点。它是我们和键盘交互的一个句柄（handle）。
- **InputDecoration**: 用于控制TextField的外观显示，如提示文本、背景颜色、边框等。
- **keyboardType**: 用于设置该输入框默认的键盘输入类型，取值如下：

TextInputType枚举值	含义
text	文本输入键盘
multiline	多行文本，需和maxLines配合使用(设为null或大于1)
number	数字；会弹出数字键盘
phone	优化后的电话号码输入键盘；会弹出数字键盘并显示"* #"
datetime	优化后的日期输入键盘；Android上会显示": -"
emailAddress	优化后的电子邮件地址；会显示"@ ."
url	优化后的url输入键盘；会显示"/ ."

`textInputAction`：键盘动作按钮图标(即回车键位图标)，它是一个枚举值，有多个可选值，全部的取值列表读者可以查看API文档，下面是当值为`TextInputAction.search`时，原生Android系统下键盘



- `style`：正在编辑的文本样式。
- `textAlign`：输入框内编辑文本在水平方向的对齐方式。
- `autofocus`：是否自动获取焦点。
- `obscureText`：是否隐藏正在编辑的文本，如用于输入密码的场景等，文本内容会用“•”替换。
- `maxLines`：输入框的最大行数，默认为1；如果为null，则无行数限制。
- `maxLength`和`maxLengthEnforcement`：`maxLength`代表输入框文本的最大长度，设置后输入框右下角会显示输入的文本计数。`maxLengthEnforcement`决定当输入文本长度超过`maxLength`时如何处理，如截断、超出等。
- `toolbarOptions`：长按或鼠标右击时出现的菜单，包括 `copy`、`cut`、`paste` 以及 `selectAll`。
- `onChange`：输入框内容改变时的回调函数；注：内容改变事件也可以通过`controller`来监

听。

- `onEditingComplete`和`onSubmitted`：这两个回调都是在输入框输入完成时触发，比如按了键盘的完成键（对号图标）或搜索键（图标）。不同的是两个回调签名不同，`onSubmitted`回调是`ValueChanged<String>`类型，它接收当前输入内容做为参数，而`onEditingComplete`不接收参数。
- `inputFormatters`：用于指定输入格式；当用户输入内容改变时，会根据指定的格式来校验。
- `enable`：如果为`false`，则输入框会被禁用，禁用状态不能响应输入和事件，同时显示禁用态样式（在其`decoration`中定义）。
- `cursorWidth`、`cursorRadius`和`cursorColor`：这三个属性是用于自定义输入框光标宽度、圆角和颜色的。

获取输入内容

获取输入内容有两种方式

第一种就是像vue、鸿蒙那种定义对应的变量然后在输入框触发 `onChange` 事件时保存输入的内容

第二种是通过 `controller` 直接获取

定义一个 `controller`：

```
1 //定义一个controller
2 TextEditingController _usernameController = TextEditingController();
```

然后设置输入框`controller`：

```
1 TextField(
2   autofocus: true,
3   controller: _usernameController, //设置controller
4   ...
5 )
```

通过`controller`获取输入框内容

```
1 print(_usernameController.text)
```

监听文本变化也有两种方式：

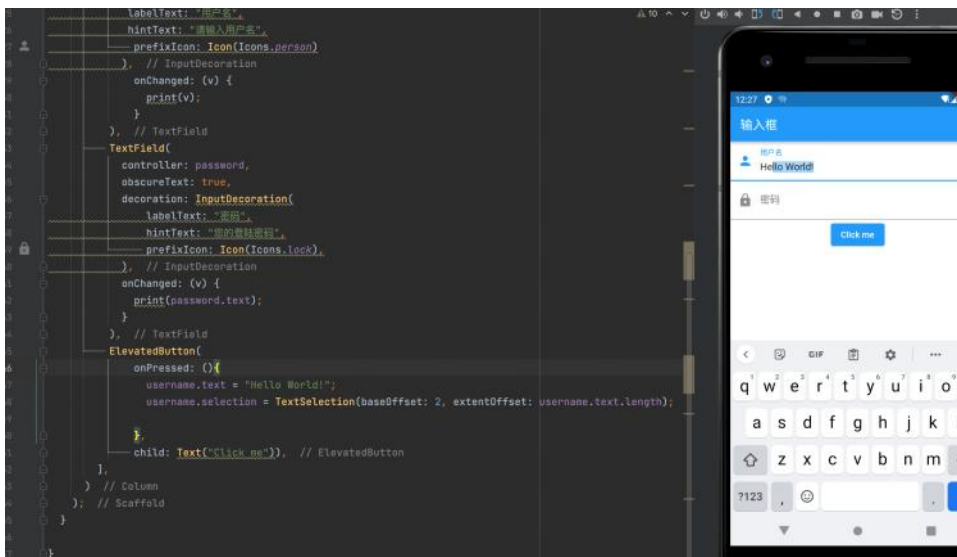
1. 设置 `onChange` 回调，如：

```
1 TextField(  
2   autofocus: true,  
3   onChanged: (v) {  
4     print("onChange: $v");  
5   }  
6 )
```

2. 通过 `controller` 监听，如：

```
1 @override  
2 void initState() {  
3   // 监听输入改变  
4   _usernameController.addListener(() {  
5     print(_usernameController.text);  
6   });  
7 }
```

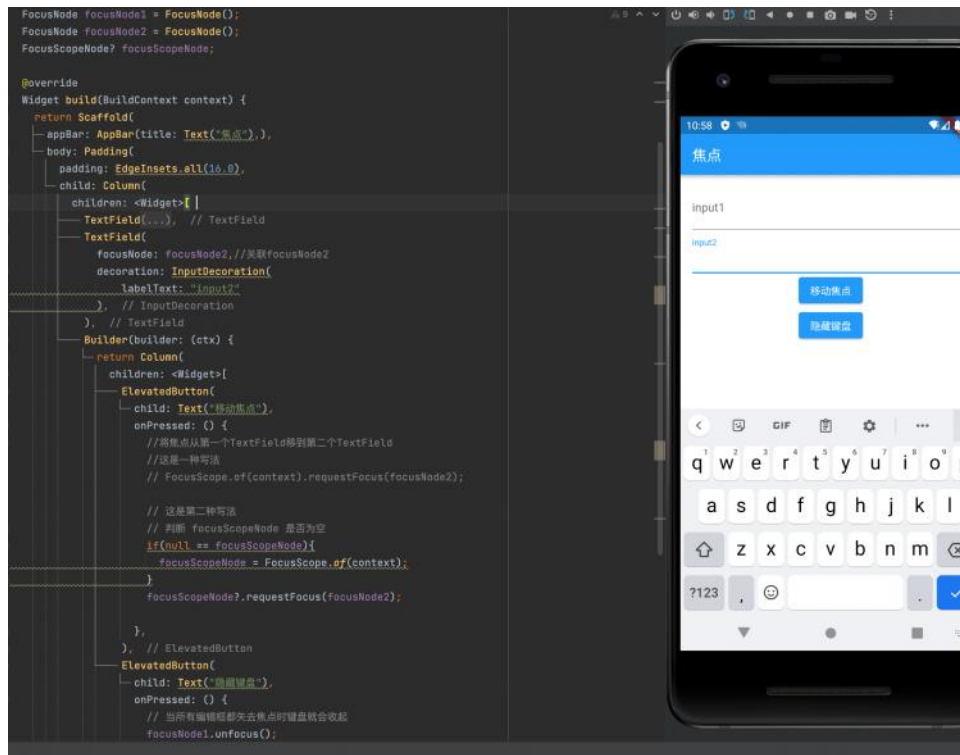
两种方式相比，`onChanged` 是专门用于监听文本变化，而 `controller` 的功能却多一些，除了能监听文本变化外，它还可以设置默认值、选择文本，下面我们看一个例子：



控制焦点

文本框的焦点可通过 `FocusNode` 和 `FocusScopeNode` 控制，默认情况焦点是由 `FocusScope` 管理，它代表焦点控制范围，可以在这个范围内通过 `FocusScopeNode` 在疏狂之间移动焦点、设置默认焦点等。

可通过 `FocusScope.of(context)` 来获取 `Widget` 树中默认的 `FocusScopeNode`



还有更多方法去看 API 文档

5) 监听焦点状态改变事件

`FocusNode` 继承自 `ChangeNotifier`，通过 `FocusNode` 可以监听焦点的改变事件，如：

```
1    ...
2    // 创建 focusNode
3    FocusNode focusNode = FocusNode();
4    ...
5    // focusNode绑定输入框
6    TextField(focusNode: focusNode);
7    ...
8    // 监听焦点变化
9    focusNode.addListener((){
10       print(focusNode.hasFocus);
11    });
```

获得焦点时 `focusNode.hasFocus` 值为 `true`，失去焦点时为 `false`。

Form表单

2024年5月9日 星期四 23:57

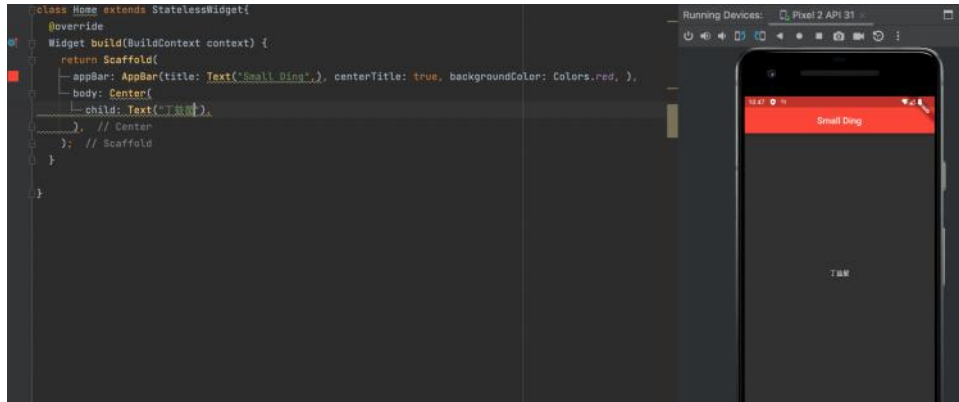
表单案例



单组件布局

2024年4月21日 星期日 22:46

Center



这个组件没有其他什么用法

线性布局

2024年4月21日 星期日

22:47

Row

```
9      return Scaffold(  
10      — appBar: AppBar(title: Text("Small Ding"), centerTit  
11      body: Row(  
12      children: [  
13      — Text("丁蕊星"),  
14      — Text("丁蕊星"),  
15      — Text("丁蕊星"),  
16      — Text("丁蕊星"),  
17      — Text("丁蕊星"),  
18      — Text("丁蕊星"),  
19      — Text("丁蕊星"),  
20      — Text("丁蕊星"),  
21      ],  
22      ), // Row  
23      ); // Scaffold  
24  }  
25  
26  }
```

Column

```
9      return Scaffold(  
10      — appBar: AppBar(title: Text("Small Ding"), centerTitle: true  
11      body: Column(  
12      children: [  
13      — Text("丁蕊星"),  
14      — Text("丁蕊星"),  
15      — Text("丁蕊星"),  
16      — Text("丁蕊星"),  
17      — Text("丁蕊星"),  
18      — Text("丁蕊星"),  
19      — Text("丁蕊星"),  
20      ],  
21      ), // Column  
22      ); // Scaffold  
23  }  
24  
25  }
```

这两种布局与鸿蒙中的布局思维基本一致，但在flutter中这当内容超过屏幕区域则会报错

Container

2024年4月24日 星期三 00:04

开发过程中使用频率最高的单组件控件，除了只能放一个子节点外类似于 div 标签

```
return Scaffold(  
  appBar: AppBar(  
    title: Text("Small Ding"),  
  ), // AppBar  
  body: Container(  
    // 外边距  
    margin: EdgeInsets.only(top: 50, left: 120),  
    width: 200,  
    height: 150,  
    // 背景样式  
    decoration: BoxDecoration(  
      gradient: RadialGradient(  
        colors: [Colors.red, Colors.orange],  
        center: Alignment.topLeft,  
        radius: 0.98  
      ), // RadialGradient  
      // 阴影  
      boxShadow: [  
        BoxShadow(  
          color: Colors.black54,  
          offset: Offset(2.0, 2.0),  
          blurRadius: 4.0,  
        ) // BoxShadow  
      ],  
    ), // BoxDecoration  
    // 卡片的倾斜变换  
    transform: Matrix4.rotationZ(.2),  
    // 卡片内的内容对其方式  
    alignment: Alignment.center,  
    child: Text(  
      "好的哥",  
      style: TextStyle(color: Colors.white, fontSize: 40),  
    ),  
  ),  
);
```

容器的大小可通过 width、height 属性来指定，也可以通过 constraints 来指定如果他们同时存在 width、height 优先，实际上 Container 内部会根据 width、height 来生成一个 constraints

Color 和 decoration 互斥，如果同时设置则会报错，同理当指定 color 时 Container 内会自动创建一个 decoration

快捷键

2024年6月9日 星期日 23:15

- 1. 快速创建Widget: 在dart文件中输入stf或stl出现提示后按回车即可
- 1. 快速修复: option + 回车
- 1. 自动生成构造函数: 选中 final 参数, 快捷键: option + 回车
- 1. 添加父组件、变为子组件、删除子组件: option+回车
- 1. 万能的搜索: 双击shift
- 1. 查看最近打开的文件: command + E
- 1. 重命名: fn+shift+f6
- 1. 查看当前类结构: command + fn + f12
- 1. 查看源码: 将光标放到要查看源码的类名或方法名上, 长按command 然后点击
- 1. 查看类的子类: 选中要查看的类, 然后: command + B 或 option + command + B
- 1. 将代码更新到模拟器上: 选中模拟器然后 command + R
- 1. 导入类的快捷键: 将光标放在要导入类的上面, 然后按 option + enter
- 1. 前进后退: 当跟踪代码的时候, 经常跳转到其他类, 后退快捷键: option+command+方向左键, 前进快捷键: option+command+方向右键
- 1. 全局搜索: command + shift + F
- 1. 全局替换: command + shift + R
- 1. 查找引用: option + shift + F7